# Software Architecture and Design Specification

## Project: Pocket Packet Sniffer

**Deliverable 3**

Prepared by: VIKAS KULAKARNI

SHIVANAND G C

SHIVAKUMARA G P

SANJITH R

## Revision History

| Version | Date | Author | Change Summary |
|---|---|---|---|
| 1.0 | 04-10-2026 | Vikas Kulakarni | Initial SASD prepared from the approved SRS and Software Test Plan. |

## Approvals

| Role | Name | Signature / Date |
|---|---|---|
| Team Lead | SHIVANAND | |
| QA / Test Lead | Sanjith R | |
| Faculty / Instructor | | |

## 1. Introduction

### 1.1 Purpose

This document explains the architecture and design we are following for Pocket Packet Sniffer. The main idea is to keep packet capture, analysis, storage and presentation as separate parts. This makes the project easier for us to build, test and extend later without changing the complete system.

### 1.2 Scope

Pocket Packet Sniffer is used to capture network packets from WiFi, Ethernet or BLE, analyze the captured packets, store the required information and show useful traffic details to the user. The main interface is a CLI. Suspicious traffic detection, multiple capture backends at the same time and a web dashboard are kept as stretch features as mentioned in our SRS.

### 1.3 Audience

This document is mainly for our project team and the faculty evaluating the project. It can also be useful for someone who wants to understand the design or add a new capture backend or protocol analyzer later.

### 1.4 Definitions

Some common terms used in this document are PCAP, monitor/promiscuous mode, BLE advertisement, Capture Layer, Analysis Layer, Storage Layer and Presentation Layer. We also use the Strategy pattern for capture backends and the Observer pattern for alert events.

# 2. Document Overview

## 2.1 How to Use This Document

The architecture section explains how the main parts of Pocket Packet Sniffer are connected. The design section goes into the actual flow between these parts using component and sequence diagrams. The later sections connect the design back to the requirements from our SRS.

## 2.2 Related Documents

- Pocket Packet Sniffer Software Requirements Specification (SRS), Version 1.3
- Pocket Packet Sniffer Software Test Plan (STP), Version 1.3
- Pocket Packet Sniffer Project Proposal
- Scapy and libpcap documentation referenced by the SRS/STP

# 3. Architecture

## 3.1 Goals & Constraints

- Maintain clear separation between Capture, Analysis, Storage, and Presentation layers.
- Allow capture backends and protocol analyzers to be extended without unnecessary changes to unrelated layers.
- Run within the resource limits of Raspberry Pi-class hardware.
- Support authorized passive network capture only; the system must not inject, modify, or replay traffic.
- Keep captured data local unless the user explicitly exports it.
- Use available capture libraries and interfaces such as Scapy/libpcap.

## 3.2 Stakeholders & Concerns

| Stakeholder | Primary Concerns |
|---|---|
| Project Team | Easy integration between modules and code that is simple to maintain |
| Evaluator / Instructor | Design matching the SRS and requirements that can be tested |
| End User | Simple CLI output, live statistics, filtering and reports |
| Developer / Maintainer | Easy addition of new capture backends and analyzers |

## 3.3 Component (UML) Diagram

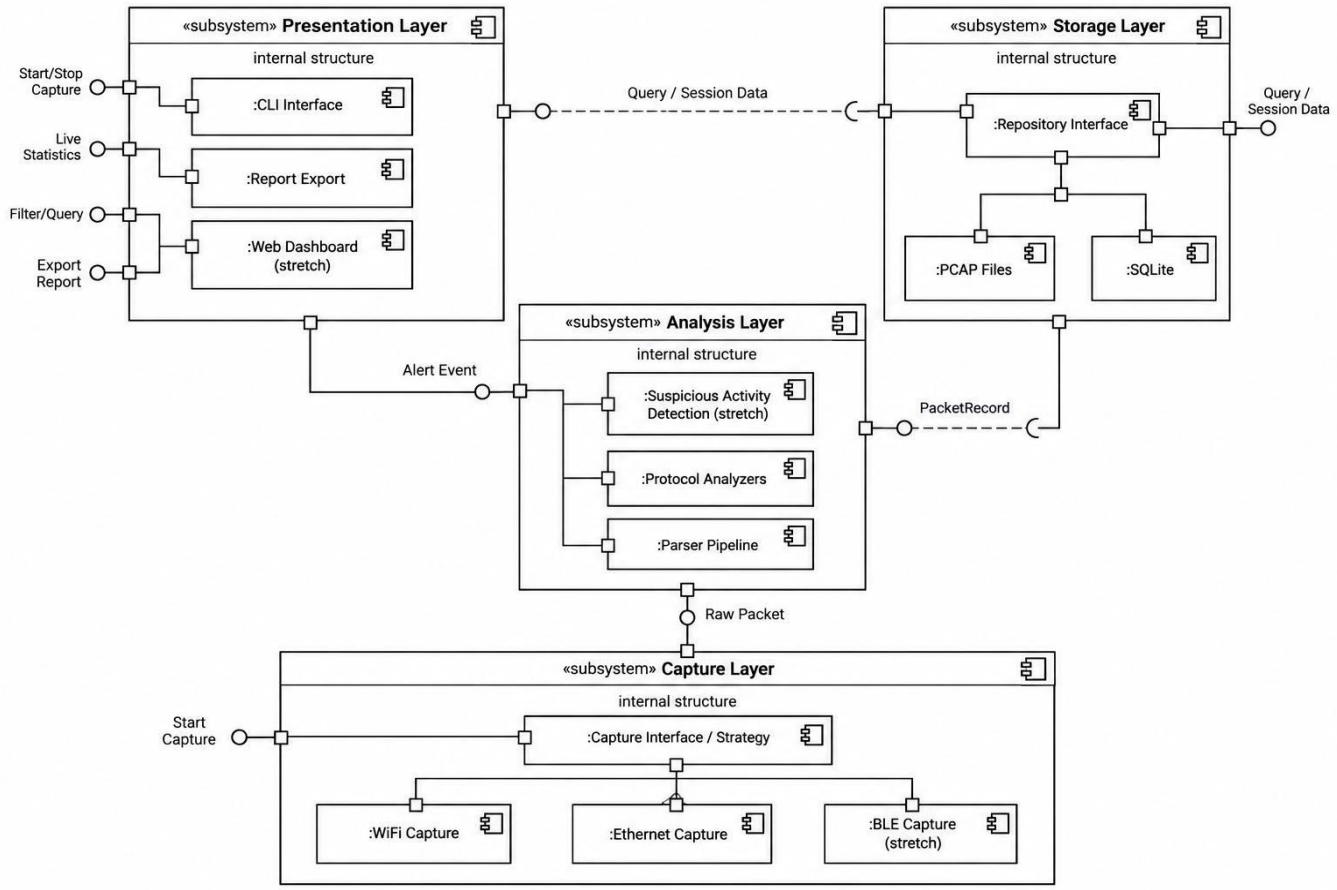



Figure 1. Pocket Packet Sniffer layered component architecture.

## 3.4 Component Descriptions

Each layer has one main responsibility. The interfaces are kept simple so that the design can be explained and implemented without making the project unnecessarily complex.

### Capture Layer

Takes a selected network interface and capture backend as input and produces raw packets. WiFi, Ethernet and BLE backends follow the same Capturer interface. BLE remains a stretch feature.

### Analysis Layer

Receives raw packets from the Capture Layer. It parses the packet, identifies the protocol and extracts the fields required by the SRS, such as timestamp, source address, destination address, protocol and packet size. Protocol-specific analyzers can be added to the analysis pipeline.

### Storage Layer

Receives the analyzed packet information and stores the capture. PCAP is used for raw packet storage and SQLite can be used for indexed metadata and queries. The Repository interface keeps this storage choice separate from the other layers.

### Presentation Layer

Provides the main CLI used by the user. It starts/stops capture, shows live packet statistics, accepts storedcapture queries and supports session report export. A web dashboard is kept as a stretch extension.

### Suspicious Activity Detection (Stretch)

Runs as part of the Analysis Layer when the stretch feature is enabled. It checks analyzed traffic against the simple suspicious patterns defined in the SRS and publishes an event when a match is found.

The important design rule is that a layer does not directly access the internal implementation of another layer. For example, the Analysis Layer does not directly open a SQLite database; it passes data through the Storage/Repository interface.

## 3.5 Chosen Architecture Pattern and Rationale

We are using a layered architecture because our project naturally has four different responsibilities: capturing packets, analyzing them, storing the results and showing them to the user. Keeping these as separate layers also matches our SRS. For the Capture Layer, we use a Strategy-based approach so WiFi, Ethernet and BLE can follow the same capture interface. The analyzers are kept as a pipeline so another analyzer can be added later without changing the existing ones. For suspicious activity alerts, an Observer-style event flow can be used so the Analysis Layer does not have to directly depend on the Presentation Layer.

## 3.6 Technology Stack & Data Stores

| Area | Technology / Choice | Purpose |
|---|---|---|
| Runtime | Python 3.x | Primary implementation environment |
| Packet Capture / Parsing | Scapy, libpcap | Packet capture and low-level parsing |
| Storage | PCAP and SQLite | Raw capture storage and indexed metadata/query support |
| Target OS / Hardware | Linux / Raspberry Pi-class hardware | Target execution environment |
| Optional Analysis | Zeek (stretch) | Deeper protocol-level analysis |
| Testing | PyTest, Scapy, Wireshark, Git | Testing, inspection and project version control |

## 3.7 Risks & Mitigations

| Risk | Mitigation |
|---|---|
| WiFi monitor mode unavailable | Use Ethernet capture as the practical fallback and keep WiFi as an interchangeable backend. |
| BLE implementation delayed | Keep BLE explicitly as a stretch backend; the core layered design does not depend on it. |
| Layer boundaries become unclear | Review interfaces regularly and treat direct cross-layer access as a design defect. |
| Sensitive captured traffic | Capture only on authorized networks and protect local capture files with appropriate permissions. |
| Capture interface or permission failure | Validate interface and permissions before capture and return clear actionable errors. |

## 3.8 Traceability to Requirements

| Requirement | Architectural Component |
|---|---|
| REQ-1 | Capture Layer / Capturer interface |
| REQ-2 | Capture Layer / interface and permission validation |
| REQ-3 | Analysis Layer / parser pipeline |
| REQ-4 | Analysis Layer / analyzer pipeline |
| REQ-5 | Storage Layer / Repository interface |
| REQ-6 | Storage Layer / Repository query operations |
| REQ-7 | Presentation Layer / live statistics |
| REQ-8 | Presentation Layer / report exporter |
| REQ-9 | Analysis → Presentation / Observer-style event flow (stretch) |

| | |
|---|---|
| REQ-10 | Capture Layer / multiple Capturer instances (stretch) |

### 3.9 Security Architecture

- Authorized-use boundary: the tool is intended only for networks/devices the user owns or is explicitly authorized to monitor.
- Passive operation: capture functionality does not inject, alter, or replay network traffic.
- Local protection: captured files may contain sensitive information and therefore require appropriate file permissions.
- Data locality: captured traffic is not silently uploaded; explicit export is required.
- Permission validation: the Capture Layer reports unavailable interfaces or insufficient permissions clearly.
- Security-sensitive events such as suspicious activity are represented as analysis events rather than tightly coupling detection to presentation.

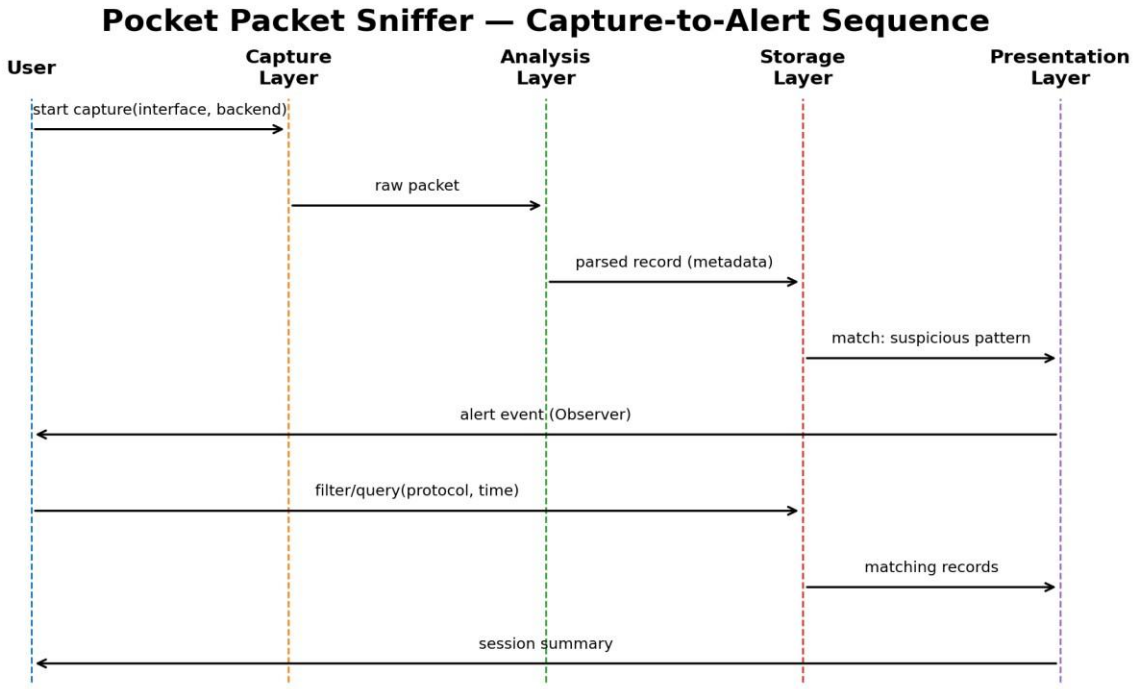
## 4. Design

### 4.1 Design Overview

The working flow starts when the user selects an interface and starts packet capture. The Capture Layer collects the raw packets and sends them to the Analysis Layer. The Analysis Layer extracts details such as source, destination, protocol, packet size and timestamp. These details can then be stored through the Storage Layer using PCAP files and SQLite. The Presentation Layer uses this information to show live statistics, query old captures and export a session summary.

### 4.2 UML Sequence Diagrams

The following two sequence diagrams show the two main flows we considered for the project. The first shows what happens from starting a capture up to displaying live statistics. The second shows how a user can search or filter packets that were already stored.




Flow follows the Capture-to-Alert sequence defined in the SRS.

Figure 2. Start capture, analyze/store packet data, and present live statistics.

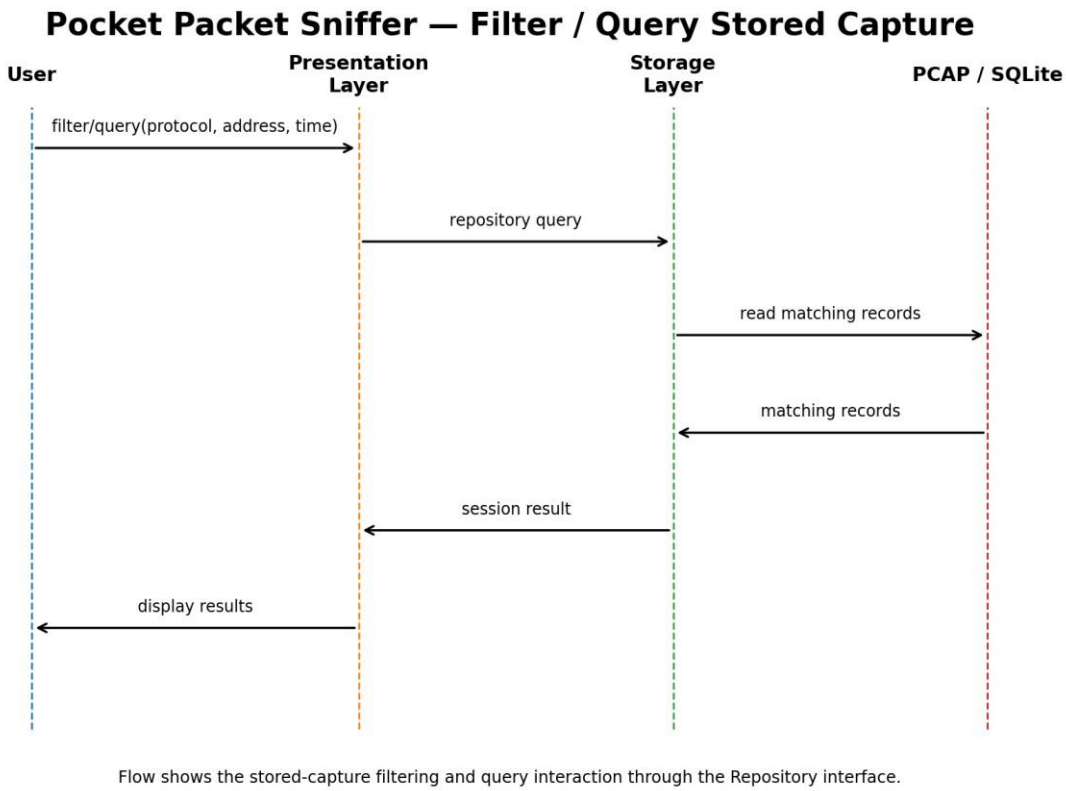



Figure 3. Filter/query stored capture records and return results to the user.

## 4.3 API / Interface Design

Our project does not expose a separate REST service in the current SRS. For the SASD, we therefore define the interfaces between the main components. These interfaces describe what each component provides to the next layer without tying the design to a particular implementation.

### 4.3.1 Capturer Interface

The Capturer interface is shared by the WiFi, Ethernet and BLE capture backends. This is the main part of the Strategy design in the Capture Layer.

start_capture(interface, backend)

Starts packet capture on the selected interface and backend.

get_packet()

Returns the next captured raw packet to the Analysis Layer. stop_capture()

Stops the current capture session and releases the capture interface.

#### 4.3.2 Repository Interface

The Repository interface keeps the Storage Layer separate from the actual PCAP or SQLite implementation. The same interface can be used by the Presentation Layer for stored-capture queries.

save(record)

Stores the packet record and associated capture information. query(protocol,

address, time_window)

Returns records matching the selected protocol, address or time range.

get_session_summary(session_id)

Returns the summary information required for a session report.

#### 4.3.3 Analysis Interface

The Analysis Layer receives raw packets from the Capture Layer and passes the required information through the parser/analyzer pipeline.

analyze(packet)

Parses the packet and identifies the relevant protocol and metadata. register_analyzer(analyzer)

Allows a new protocol analyzer to be added without changing the existing analyzers.

#### 4.3.4 Alert Interface (Stretch)

For suspicious activity detection, the Analysis Layer can publish an alert event. The Presentation Layer can subscribe to these events instead of being directly coupled to the detection logic.

subscribe(listener)

Registers a component that wants to receive alert events.

publish(event)

Publishes a suspicious-activity event to subscribed listeners.

### 4.4 Error Handling, Logging & Monitoring

The error handling is kept simple and local to the component that can detect the problem. The goal is to fail gracefully and tell the user what needs to be checked.

### Capture errors

- If the selected interface does not exist, the Capture Layer stops the request and reports that the interface could not be found.
- If the program does not have the required capture permission, the system reports a permission error instead of continuing with an invalid capture.
- If a capture backend cannot start, the capture session is not started and the user receives a clear error message.

### Analysis errors

- If a packet cannot be parsed by an analyzer, the system should handle that packet without bringing down the complete capture process.
- Analyzer-specific problems should remain inside the Analysis Layer and should not require changes to the Storage or Presentation layers.

### Storage errors

- If a capture cannot be saved or queried, the Storage Layer returns an error to the Presentation Layer so that the user can be informed.
- The system should not silently discard a storage failure or report a successful save when the operation failed.

### Logging and monitoring

- Logs should contain useful operational information such as capture start/stop, interface errors and storage failures, but should avoid unnecessarily printing sensitive packet contents.
- The live interface can show packet rate, protocol breakdown and the top five talkers, which are the statistics required by the SRS.
- The STP also identifies packet loss, refresh time, test execution results and defect information as useful project-level metrics.

## 4.5 UX Design

- The primary user interface is the command-line tool.
- The CLI supports capture start/stop, live statistics, stored-capture filtering/querying, and session report export.
- Live output should show packet rate, protocol breakdown, and top five talkers during active capture.
- Errors should state what went wrong and what the user should check next instead of exposing only a raw stack trace.
- A web dashboard may provide equivalent views as a stretch extension.

## 4.6 Open Issues & Next Steps

- Complete and validate the capture backend(s) supported by the implementation environment.
- Complete code-level RTM entries once implementation filenames are finalized.
- Validate packet parsing and analyzer extensibility with representative TCP, UDP, ARP, ICMP and other authorized traffic.
- Implement or validate suspicious activity detection as the SRS stretch feature if time permits.
- Evaluate multi-backend capture as the SRS stretch feature after the single-backend flow is stable.
- Evaluate web dashboard and Zeek integration only after core requirements are stable.

**Design explanation in one line:** capture the packet → analyze it → store the useful information → show the result to the user.

# 5. Appendices

## 5.1 Glossary

| Term | Meaning |
|---|---|
| PCAP | Standard file format for storing captured network packets. |

| | |
|---|---|
| Promiscuous / Monitor Mode | Network interface mode used to observe traffic beyond packets addressed directly to the capturing device, subject to hardware/driver support. |
| BLE Advertisement | Periodic broadcast packets sent by Bluetooth Low Energy devices that can be observed without pairing. |
| Strategy Pattern | Design in which interchangeable capture backends implement a common interface and are selected at runtime. |
| Observer Pattern | Design in which interested components subscribe to events generated by another component. |
| PacketRecord | Logical representation of extracted packet metadata used between analysis, storage and presentation concerns. |

## 5.2 References


- Pocket Packet Sniffer SRS, Version 1.3.
- Pocket Packet Sniffer Software Test Plan, Version 1.3.
- Pocket Packet Sniffer Project Proposal.
- Scapy documentation.
- Zeek documentation.
- libpcap documentation.


## 5.3 Tools

For diagrams we can use tools such as draw.io or PlantUML. The main implementation and testing tools already selected for the project are Python, Scapy, libpcap, SQLite, PyTest, Wireshark and Git.